SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA0609	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	Yuvasri.B	**Reg. No.:**	192421231
Section / Batch:	BTech IT	**Date:**	19/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I Yuvasri .B /192421231 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Yuvasri B

Register No: 192421231

Student: YUVASRI B

Assigned Question:

Q40. Debugging Insertion Sort Code (Incorrect Final Placement Index)

The following Insertion Sort implementation performs element shifting, but the final insertion step uses the wrong index and causes misplaced values in the sorted prefix.

Carefully analyze the shifting process and identify the exact source of the indexing error. Debug the code step-by-step so that each element is inserted into its correct position after all larger elements are shifted. Optimize the implementation for readability and maintain consistent variable usage. Analyze the corrected algorithm in terms of time complexity and rewrite the final clean version with proper comments. def insertion_sort(arr): for i in range(1, len(arr)):

key = arr[i] j = i - 1 while j >= 0 and arr[j]

> key:

arr[j+1] = arr[j] j -= 1

arr[j] = key # ERROR

return arr

Output:

Error:

Wrong insertion index.

Correct: arr[j+1] = key

Debug the Program: Replace

arr[j] = key with arr[j+1] = key

Corrected outuput:

Error Identified:

- The insertion statement used arr[j] = key, which inserts the key at the wrong index.

Correction:

- Replaced arr[j] = key with arr[j + 1] = key.

Optimization:

- Added comments for better readability.
- Maintained proper indentation.
- Used consistent variable names (key, j, arr).
- The algorithm remains efficient and easy to understand.

Time Complexity:

- Best Case: **O(n)**
- Average Case: **O(n²)**
- Worst Case: **O(n²)**
- Space Complexity: **O(1)**

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, wellstructured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:

Signature: _____ Date: _____	Signature: _____ Date: _____	Signature: _____ Date: _____